

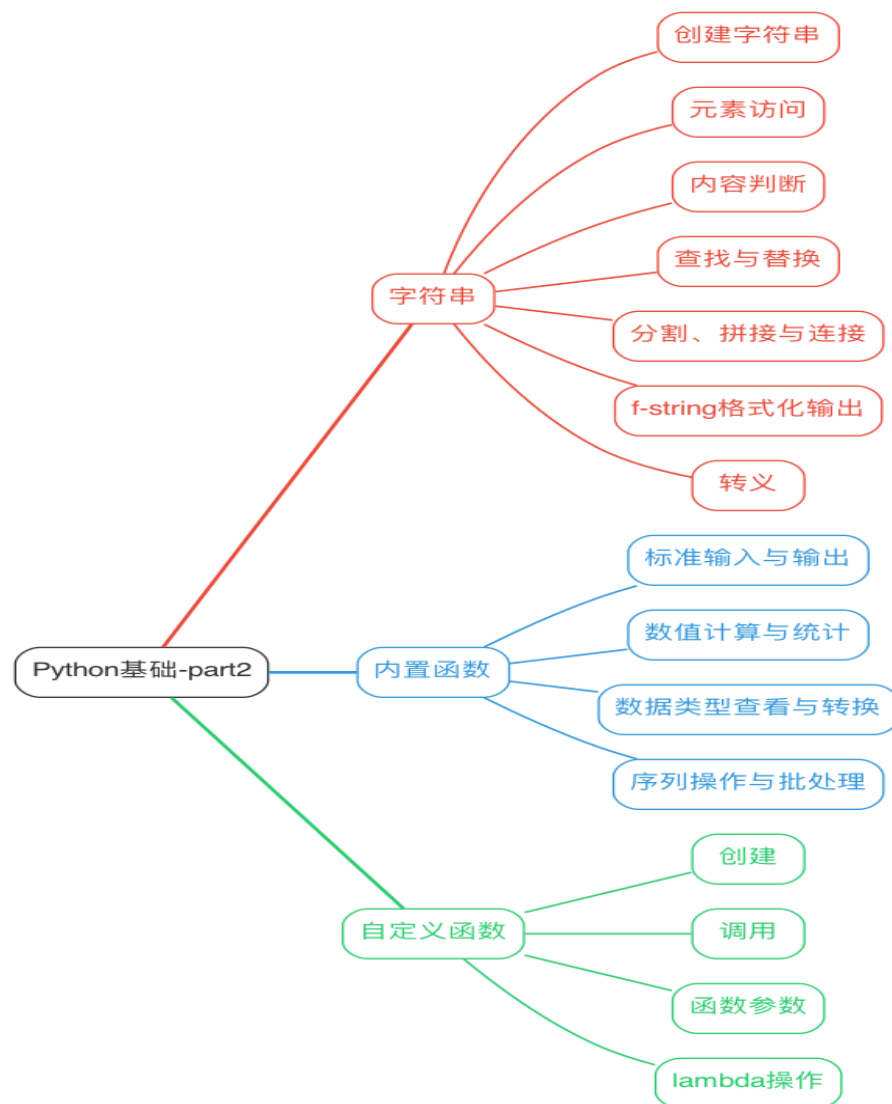


《实用Python编程》

第3讲：Python基础-part2（字符串与函数）

实用Python编程 教研组

提纲



从一个文本数据清洗的例子开始

- 如何将格式混乱的账单整理为结构清晰的记录?

```
data = [" 2026-08-25_ID98453_John Doe _MEAL_  
$128.50USD ",  
"2026/08/26-ID10247-Marta- hotel -$450.00USD",  
"2026.08.26_id00382_ ALEX _flight_$1,200.00 USD"]
```



```
('98453', '2026-08-25', 'John Doe', 'MEAL', 128.5)  
( '10247', '2026-08-26', 'Marta', 'hotel', 450.0)  
( '00382', '2026-08-26', 'ALEX', 'flight', 1200.0)
```



字符串 (str)

- 一个字符串是由字符组成的一个序列
 - 以引号开头，引号结尾。可以是单引号或双引号，但前后引号必须配
 - `s1 = "hello python"`
 - `s2 = 'helloPython'`
- 字符串的长度
 - 引号中间包含的字符数（含空白字符）
 - 长度为零的字符串称为**空串**
- 字符串可视为以**字符为元素（但内容不可变）**的列表
 - 因此，列表索引和切片操作同样也适用于字符串
 - 尝试将首字母改为大写（`s1[0] = 'H'`），**会报错！**

```
s1 = "hello Python"
s1[0] = "H"

TypeError                                 Traceback (most recent call last)
Cell In[17], line 2
     1 s1 = "hello Python"
----> 2 s1[0] = "H"

TypeError: 'str' object does not support item assignment
```



字符串操作

- 创建

```
# 字符串创建
s0 = '' # 空串，以单引号开头和结尾
s1 = "" # 空串，以双引号开头和结尾
s2 = '中国人民大学'
s3 = "中国 人民 大学"
print(len(s0), len(s1), len(s2), len(s3)) # 输出 0 0 6 8
```



字符串操作

- 元素访问
 - 索引、切片

```
1 # 字符串创建
2 s0 = '' # 空串, 以单引号开头和结尾
3 s1 = "" # 空串, 以双引号开头和结尾
4 s2 = '中国人民大学'
5 s3 = "中国 人民 大学"
6 print(len(s0), len(s1), len(s2), len(s3)) # 输出 0 0 6 8
7
8 # 访问单个元素
9 print(s2[2]) # 正向索引, 从0开始, 输出"人"
10 print(s2[-1]) # 反向索引, 从-1开始, 输出"学"
11
12 # 访问多个元素
13 print(s2[2:]) # 获取第2个及以后的字符串, 输出"民大学"
14 print(s2[2:4]) # 获取第2到第4的字符串, 输出"人民"
15 print(s2[::2]) # 步长为2, 输出"国民学"
16 print(s2[::-1]) # 步长为1, 反向输出字符串, 输出"学大民人国中"
```



字符串操作

• 内容判断

表 2.4: 用于字符串内容判断的内置方法

方法名	功能描述	返回为 True 的示例
基础类型判断:		
<code>isalnum</code>	检查是否全为字母或数字（含中文字符）	<code>"数据123".isalnum()</code>
<code>isalpha</code>	检查是否全为字母（含中文字符）	<code>"Python分析".isalpha()</code>
<code>isdigit</code>	检查是否全为阿拉伯数字字符	<code>"123".isdigit()</code>
<code>isnumeric</code>	检查是否全为 Unicode 数字字符（含中文数字、罗马数字）	<code>"五五伍".isnumeric()</code>
<code>isspace</code>	检查是否全为空白字符（空格、制表符、换行符）	<code>"\t \n".isspace()</code>
存在性判断:		
<code>str1 in str2</code>	检查 str1 是否为 str2 的子串	<code>"人民" in "中国人民大学"</code>
开头/结尾判断:		
<code>str1.startswith(str2)</code>	检查 str1 是否以 str2 开头	<code>"中国人民大学".startswith("中国")</code>
<code>str1.endswith(str2)</code>	检查 str1 是否以 str2 结尾	<code>"ruc.jpg".endswith(".jpg")</code>



课堂练习-2-2-1

编写一个判断口令强度的程序

- 给定一个口令，当它同时满足下列所有条件时，则认为是强口令，否则就是弱口令
 - 规则1：口令必须同时包含大小写英文字母、数字和特殊字符（下划线、叹号、冒号）
 - 规则2：口令长度不小于8

```
1 pwd_list ["Alpha99", "8848", "admin_super_user", "admin_Super_user"]
2 for pwd in pwd_list:
3     # to-do: 强度判断
4     # ...
5     # ...
6     if okay:
7         print(pwd, 'is okay')
8     else:
9         print(pwd, 'is not okay')
```

字符串操作

- `str1.find(str2[,start,end])`
 - 在str1 (主串) 中查找str2 (模式串)
 - 可选参数start, end为查找的起点和终点。不指定则从头到尾查找
- `s.replace(old, new[,count])`
 - 将s中的子串old替换为new
 - 可选参数count指定最多替换次数。不指定则全部替换

```
1 s = "DeepSeek第六讲: DeepSeek人工智能与可信数据空间构建"
2 # 查找
3 print(s.find("DeepSeek")) # 不限定范围, 输出0
4 print(s.find("Deep", 10)) # 限定范围, 输出12
5 print(s.find("ChatGPT")) # 不存在, 输出-1
6 # 替换
7 print(s.replace("DeepSeek", "ChatGPT")) # 输出 ChatGPT第六讲: ChatGPT人工智能与可信数据空间构建
8 print(s.replace("DeepSeek", "ChatGPT", 1)) # 输出 ChatGPT第六讲: DeepSeek人工智能与可信数据空间构建
9 print(s.replace("deepseek", "ChatGPT")) # 输出 DeepSeek第六讲: DeepSeek人工智能与可信数据空间构建
```

字符串操作

- 分割(split): 将字符串按指定分隔符拆分成一个字符串列表
- 拼接(join): 将一个字符串列表或集合用指定连接符拼接成一个字符串
- 连接(concatenation): 直接用加号 (+) 拼接多个字符串
 - 对同一个字符串进行重复拼接时, 可以用乘法操作

```
1 text = "中国人民大学,北京|重庆大学,重庆|浙江大学,杭州"
2 # 先按 "|" 分割每条记录
3 records = text.split("|")
4 # 遍历每条记录, 再按 "," 分割字段
5 for record in records:
6     univ, city = record.split(",")
7     out = "".join([univ, "位于", city])
8     out1 = f"{univ}位于{city}" # out1是out的f-string实现
9     print(out)
10    print("-"*20)
```



字符串操作

f-string格式化输出

- 只要在字符串的引号前加上f作为前缀，就可以在字符串内部用 {} 直接塞入变量、表达式甚至调用函数

```
1 last_code = '1001'
2 location = '办公室门口'
3 contact = '15611112222'
4 message = f'【京东快递】快递尾号{last_code}已放在{location}，快递员电话{contact}'
5 print(message) # 【京东快递】快递尾号1001已放在办公室门口，快递员电话15611112222
```

【京东快递】快递尾号1001已放在办公室门口，快递员电话15611112222

控制小数位、高位补齐

```
1 item_list = [('苹果', 12.125), ('牛奶', 9.9), ('杂粮面包', 5.16)]
2 for i, item in enumerate(item_list):
3     print(f'[{i:03d}] {item[0]} 单价: {item[1]:.1f}')
```

```
[000] 苹果 单价: 12.1
[001] 牛奶 单价: 9.9
[002] 杂粮面包 单价: 5.2
```

:03d 代表将整数 (decimal) 格式化为 3 位宽度，如果数字本身不足 3 位，前面用 0 补齐

:.1f 代表将浮点数 (float) 保留 1 位小数



字符串操作

转义 (escape) : 改变字符的固有含义

- 在字符串中加入无法直接输入的字符 (比如“换行”)
- 让具有特殊功能的字符回归本义 (比如引号作为引号而不是分界)

表 2.5: Python 常见转义符

转义符	功能描述	示例代码	示例效果
\n	换行	"第一行xxx\n第二行yyy"	第一行 xxx 第二行 yyy
\t	制表符 (4 个空格宽度)	"姓名\t年龄"	姓名 年龄
\uXXXX	Unicode 字符 (4 位十六进制)	"\u4e2d", "1\u4e2d2"	中, 1 中 2
\", \'	将引号当作普通字符	"人大是\"985工程\"重点建设大学"	人大是 “985 工程” 重点建设大学
\\	将反斜杠作为普通字符	"C:\\Users"	C:\Users



课堂练习-2-2-2

- 修复代码，使字符串能正常输出 Let's go to the "Python" class!

```
1 msg1 = 'Let's go to the "Python" class!'  
2 print(msg1)
```

Cell In[33], line 1

```
msg1 = 'Let's go to the "Python" class!'
```

^

SyntaxError: unterminated string literal (detected at line 1)

函数

- 函数(function)
 - 一段被命名的、用于完成特定任务的代码块
 - 使用函数可以让程序更加简洁、易懂、易维护
- 主要用途
 - 代码复用：一次编写，到处使用，避免重复劳动
 - 模块化求解：将复杂任务拆解为多个函数协作
 - 参数化操作：通过输入参数动态调整程序行为

内置函数

- 内置函数(built-in functions)是Python解释器预先定义好的、无需导入任何模块即可直接使用的函数
- 提供了最基础且高频的操作
 - 标准输入与输出 `input`, `print`
 - 数值计算与统计 `min`, `max`, `sum`, ...
 - 数据类型查看与转换 `type`, `int`, `float`, `str`, ..
 - 序列操作与批处理 `len`, `range`, `any`, `all`, `map`, `zip`, ...

```
import builtins
print(dir(builtins))
```

内置函数

- 标准输入与输出
 - 键盘输入、屏幕输出

```
while True:
    val = input("请输入数字 (逢双结束) : ")
    print(f"你输入的是{val}")
    val = int(val)
    if val%2==0:
        break
print('The End')
```

请输入数字 (逢双结束) : 101

你输入的是101

请输入数字 (逢双结束) : 5

你输入的是5

请输入数字 (逢双结束) :

输入框，按回车完成输入



图 2.2: 标准输入函数input使用演示



内置函数

- 数值计算与统计

表 2.6: Python 内置的数值计算与统计函数

函数	功能	示例
<code>round</code>	银行家舍入法	<code>round(3.14159), round(3.14159, 2)</code> → 3, 3.14
<code>abs</code>	返回绝对值	<code>abs(-5)</code> → 5
<code>pow</code>	幂运算	<code>pow(2, 3)</code> → 8
<code>sum</code>	对可迭代对象（多元组、列表、集合）求和	<code>sum([1, 2, 3])</code> → 6
<code>min</code>	返回可迭代对象中的最小元素值	<code>min([1, 2, 3])</code> → 1
<code>max</code>	返回可迭代对象中的最大元素值	<code>max([1, 2, 3])</code> → 3

内置函数

• 数据类型查看与转换

表 2.7: Python 内置的数据类型查看与转换函数

函数	功能	示例
<code>type</code>	查看对象的数据类型	<code>type('hello')</code> , <code>type({})</code> → <code><class 'str'></code> , <code><class 'dict'></code>
<code>isinstance</code>	判断对象是否属于某个数据类型	<code>isinstance(5,int)</code> → <code>True</code>
<code>int</code>	将浮点数或数字字符串转换为整数	<code>int(1.6)</code> , <code>int("123")</code> → <code>123</code> , <code>1</code>
<code>float</code>	将整数或数字字符串转换为浮点数	<code>float(1)</code> , <code>float("1.23")</code> → <code>1.0</code> , <code>1.23</code>
<code>str</code>	将其他数据类型转换为字符串	<code>str(1)</code> , <code>str(1.23)</code> → <code>'1'</code> , <code>'1.23'</code>
<code>list</code>	将可序列化对象转换为列表	<code>list('aaabc')</code> → <code>['a', 'a', 'a', 'b', 'c']</code>
<code>set</code>	将可序列化对象转换为集合	<code>set('aaabc')</code> → <code>{'a', 'b', 'c'}</code>
<code>dict</code>	将二元组序列转换为字典	<code>dict([(1,2),(1,3)])</code> → <code>{1:3}</code>
<code>dir</code>	列出对象的所有属性与方法	<code>dir(str)</code> → <code>['__add__', ..., 'endswith', ..., 'strip', ..., 'zfill']</code>

内置函数

• 序列操作与批处理

表 2.8: Python 内置的序列操作函数

函数	功能	示例
<code>len</code>	返回对象的长度/元素个数	<code>len('hello'), [1,2] → 5, 2</code>
<code>range</code>	生成一个步进序列（用于for循环）	<code>list(range(1,6)), list(range(1,6,2)) → [1,2,3,4,5], [1,3,5]</code>
<code>enumerate</code>	同时获取序列中元素的索引和值（用于for循环）	<code>list(enumerate([1,2,3])) → [(0,1),(1,2),(2,3)]</code>
<code>sorted</code>	返回排序后的新列表	<code>sorted([3,1,2]) → [1,2,3]</code>
<code>reversed</code>	返回反转后的新序列	<code>list(reversed([1,2,3])) → [3,2,1]</code>
<code>any</code>	判断序列是否存在真值	<code>any([True,False]), any([False,False]) → True, False</code>
<code>all</code>	判断序列中元素是否全为真值	<code>all([True,True]), all([True,False]) → True, False</code>
<code>map</code>	对序列中的每一个元素执行相同的操作	<code>list(map(str,[1,2,3])) → ['1','2','3']</code>
<code>zip</code>	将多个列表中的元素对应打包成一个多元组序列	<code>list(zip(['Tom','Jerry'],['Cat','Mouse'])) → [('Tom','Cat'),('Jerry','Mouse']</code>



自定义函数

自定义函数 (user-defined functions)

- 由自己编写、具有特定功能的代码块
- 将复杂逻辑封装起来
- 方便代码复用
- 让主程序变得结构清晰、易于维护

创建自定义函数

- 从结构上分为函数头和函数体
 - 函数头(function header): 函数名 + 参数列表
 - 函数名: 跟在`def`之后, 遵循变量命名规则
 - 参数(可选): 函数执行时所需的外部数据, 放在括号内
 - 参数之间以逗号隔开, 即使没有参数, 括号也要保留
 - 函数体(function body): 缩进、执行具体逻辑的代码块
 - 返回值(可选): 函数执行完毕后返回给调用者的值, 通过`return`关键字返回

```
1 def 函数名(参数1, 参数2, ...):  
2     """  
3     总的注释(可选): 解释函数的作用  
4     """  
5     # 函数体: 执行具体逻辑的代码  
6     代码块  
7     return 返回值 # 可选: 将结果返回给调用者
```



创建自定义函数

- 类型提示(type hints)
 - 在函数头中明确标注参数和函数返回值的数据类型
 - 能使代码更容易理解

示例代码 2.28 带类型提示的自定义函数语法

```
1 def 函数名(参数1: 参数1类型, 参数2: 参数2类型, ...) -> 返回值类型:  
2     """  
3     总的注释 (可选): 解释函数的作用  
4     """  
5     # 函数体: 执行具体逻辑的代码  
6     代码块  
7     return 返回值 # 可选: 将结果返回给调用者
```

调用自定义函数

- 函数只有在被调用时，其函数体内的代码才会被执行

```
1 def greet():  
2     print("Hello!")  
3  
4 def func_with_bug():  
5     a = a + 1  
6  
7 # 调用函数  
8 greet() # 输出: Hello!
```

变量 `a` 在使用时
尚未被赋值。但该函数未被
调用，也就不会报错

调用自定义函数

- 调用带参数的函数

```
1 def greet1(name):
2     return f"Hello, {name}"
3
4 def greet2(name: str) -> str:
5     return f"Hello, {name}"
6
7 def greet3(name, title='Dr.'):
8     return f"Hello, {title} {name}"
9
10 def greet4(name: str, title: str='Dr.') -> str:
11     return f"Hello, {title} {name}"
12 # 调用函数
13 print(greet1("Watson")) # 输出: Hello, Watson
14 print(greet2("Watson")) # 输出: Hello, Watson
15 print(greet3("Watson")) # 输出: Hello, Dr. Watson
16 print(greet4("Watson")) # 输出: Hello, Dr. Watson
17 print(greet4("Watson", 'Mr.)) # 输出: Hello, Mr. Watson
18 print(greet4('Mr.', "Watson")) # 输出: Hello, Watson Mr.
19 print(greet4(title="Mr.", name="Watson")) # 输出: Hello, Mr. Watson
```



课堂练习-2-2-3

一个牧场每天需要根据动物的种类、体重以及天气情况，计算出它们当天该吃多少饲料。请编写一个名为 `calculate_feed` 的自定义函数，实现一键计算。

- 核心计算逻辑：基础饲料量为体重的5%。如果是下雨天，动物不外出活动，消耗变小，基础饲料量需要扣减 0.5 公斤
- 函数内部要打印一行提示信息，格式为："【牧场广播】正在为 动物名 计算配方..."
- 请根据任务设计函数的输入接口（参数）、输出接口（返回值），并给出测试用例

函数参数

- 形式参数(parameters)
 - 在函数头中列出的参数名，形式上描述了如何给函数传递数据
- 实际参数(arguments)
 - 在函数调用时传递的实际值

示例代码 2.32 形参与实参同名（但相互独立）

```
1 def double_func(a): # 此处的a为形参
2     a *= 2
3     print(a) # 输出2002
4
5 a = 1001 # 此处的a为实参
6 double_func(a)
7 print(a) # 输出1001
```

在double_func中对形参a的修改不影响实参a

变量存的是其所指对象的内存地址，而不是该对象的值（2.1.1节）



迷你函数：lambda操作

- 当函数功能可以通过单行表达式实现时，可以使用lambda操作
 - 一种匿名、更为简洁的自定义函数使用方式

示例代码 2.33 利用lambda操作实现列表的多键排序

语法

lambda 参数1,参数2,...: 表达式

```
1 data = [  
2     {'name': 'Alice', 'age': 25, 'score': 90},  
3     {'name': 'Bob', 'age': 30, 'score': 95},  
4     {'name': 'Charlie', 'age': 25, 'score': 95},  
5     {'name': 'David', 'age': 30, 'score': 80}  
6 ]  
7 # 先按score降序，score相同情况下再按age升序  
8 sorted_data = sorted(data, key=lambda x: (-x['score'], x['age']))  
9 for item in sorted_data:  
10     print(item)  
11 ''' 输出:  
12 {'name': 'Charlie', 'age': 25, 'score': 95}  
13 {'name': 'Bob', 'age': 30, 'score': 95}  
14 {'name': 'Alice', 'age': 25, 'score': 90}  
15 {'name': 'David', 'age': 30, 'score': 80}  
16 '''
```



课堂练习-2-2-4

现有一组店铺数据，分别记录了店铺名称, 配送时间(分钟), 人均消费(元), 店铺评分等四项。

```
shops = [  
    ["麦当劳", 25, 35, 4.7],  
    ["沙县小吃", 30, 15, 4.2],  
    ["海底捞火锅", 50, 120, 4.9],  
    ["轻食沙拉日记", 15, 45, 4.5]  
]
```

请找出配送时间最短的店铺。

- 若直接调用 `min(shops)`，会返回什么？为什么？